Projeto e Análise de Algoritmos

Recorrências

Universidade Federal do Amazonas
Instituto de Computação

Recorrências

- A expressão:

$$T(n) = \begin{cases} c & n = 1 \\ 2T\left(\frac{n}{2}\right) + cn & n > 1 \end{cases}$$

é uma *recorrência*.

- Recorrência: uma equação que descreve uma função em termos de seus valores para instâncias menores

Exemplos de Recorrências

$$s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases} \qquad s(n) = \begin{cases} 0 & n = 0 \\ n + s(n-1) & n > 0 \end{cases}$$

$$T(n) = \begin{cases} c & n = 1 \\ 2T\left(\frac{n}{2}\right) + c & n > 1 \end{cases} \qquad T(n) = \begin{cases} c & n = 1 \\ aT\left(\frac{n}{b}\right) + cn & n > 1 \end{cases}$$

Solução de Recorrências

- Métodos
  - Iteração
  - Árvore de Recursão
  - Mestre
  - Aniquilador (solução de EDOs)
  - Subsitituição

Método da Iteração

Método da Iteração

- Consiste em:
  - Expandir a recorrência algumas vezes
  - Tentar encontrar um padrão nas expansões.
  - Generaliza-las e, em geral, encontrar um somatório a ser resolvido.
  - Resolver o somatório

Ex. 1 – Iteração

- Resolver

$$s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases}$$

Ex. 1 – Iteração (2)

- s(n) = c + s(n-1)
  = c + c + s(n-2)
  = c + c + c + s(n-3)
  …
  = kc + s(n-k)

$$s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases}$$

Ex. 1 – Iteração (3)

- Até agora para k < n temos
  - s(n) = ck + s(n-k) | Isso para quando ?
- Para em s(0), logo: n-k=0 e k=n !
  - s(n) = cn + s(0) = cn
- Portanto
  - S(n) = cn

$$s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases}$$

Ex. 2 – Iteração

- Resolver

$$s(n) = \begin{cases} 0 & n = 0 \\ n + s(n-1) & n > 0 \end{cases}$$

Ex. 2 – Iteração (2)

$$s(n) = \begin{cases} 0 & n = 0 \\ n + s(n-1) & n > 0 \end{cases}$$

- s(n)

= n + s(n-1)

= n + n-1 + s(n-2)

= n + n-1 + n-2 + s(n-3)

= n + n-1 + n-2 + n-3 + s(n-4)

= …

= n + n-1 + n-2 + n-3 + … + n-(k-1) + s(n-k)

$$= \sum_{i=n-k+1}^{n} i \quad + \quad s(n-k)$$

Ex. 2 – Iteração (3)

- Até agora para k<n temos

$$\sum_{i=n-k+1}^{n} i \quad + \quad s(n-k)$$

- E se k=n ?

$$\sum_{i=1}^{n} i \quad + \quad s(0) \quad = \quad \sum_{i=1}^{n} i \quad + \quad 0 \quad = \quad n\frac{n+1}{2}$$

- Portanto:

$$s(n) \quad = \quad n\frac{n+1}{2}$$

Ex. 3 – Iteração

- Resolver

$$T(n) = \begin{cases} c & n = 1 \\ 2T\left(\frac{n}{2}\right) + c & n > 1 \end{cases}$$

Ex. 3 – Iteração (2)

- $T(n) = 2T(n/2) + c$
  $= 2(2T(n/2/2) + c) + c$
  $= 2^2T(n/2^2) + 2c + c$
  $= 2^3T(n/2^3) + 2^2 c + 2c + c$
  $= 2^4T(n/2^4) + 2^3c + 2^2c + 2^1 c + 2^0 c$
  $= \ldots$
  $= 2^kT(n/2^k) + 2^{k-1}c + 2^3c + 2^2c + 2^1 c + 2^0 c$

$$T(n) = 2^k T(n/2^k) + c\sum_{i=0}^{k-1} 2^i$$

$$T(n) = \begin{cases} c & n = 1 \\ 2T\left(\frac{n}{2}\right) + c & n > 1 \end{cases}$$

Ex. 3 – Iteração (3)

- Até agora para $2^k < n$ temos
  - $T(n) = 2^k T(n/2^k) + (2^k - 1)c$
- Para em T(1). $n/2^k = 1$; $k = \log_2 n$
  - $T(n) = 2^{\log n}\, T(n/2^{\log n}) + (2^{\log n} - 1)c$

$= n\, T(n/n) + (n - 1)c$

$= n\, T(1) + (n-1)c$

$= nc + (n-1)c = (2n - 1)c$

$$T(n) = \begin{cases} c & n = 1 \\ 2T\left(\frac{n}{2}\right) + c & n > 1 \end{cases}$$

Ex. 4 – Iteração

- Resolver

$$T(n) = \begin{cases} c & n = 1 \\ aT\left(\frac{n}{b}\right) + cn & n > 1 \end{cases}$$

Ex. 4 – Iteração (2)

$$T(n) = \begin{cases} c & n = 1 \\ aT\left(\frac{n}{b}\right) + cn & n > 1 \end{cases}$$

- T(n) =

  aT(n/b) + cn

  a(aT(n/b/b) + cn/b) + cn

  $a^2T(n/b^2)$ + cna/b + cn

  $a^2T(n/b^2)$ + cn(a/b + 1)

  $a^2(aT(n/b^2/b) + cn/b^2)$ + cn(a/b + 1)

  $a^3T(n/b^3)$ + cn(a^2/b^2) + cn(a/b + 1)

  $a^3T(n/b^3)$ + cn(a^2/b^2 + a/b + 1)

  …

  $a^kT(n/b^k)$ + cn(a^{k-1}/b^{k-1} + a^{k-2}/b^{k-2} + … + a^2/b^2 + a/b + 1)

Ex. 4 – Iteração (3)

- Assim, temos
  - $T(n) = a^k T(n/b^k) + cn(a^{k-1}/b^{k-1} + \ldots + a^2/b^2 + a/b + 1)$
- Para $k = \log_b n$
  - $n = b^k$
  - $T(n) = a^k T(1) + cn(a^{k-1}/b^{k-1} + \ldots + a^2/b^2 + a/b + 1)$
    $= a^k c + cn(a^{k-1}/b^{k-1} + \ldots + a^2/b^2 + a/b + 1)$
    $= ca^k + cn(a^{k-1}/b^{k-1} + \ldots + a^2/b^2 + a/b + 1)$
    $= cna^k/b^k + cn(a^{k-1}/b^{k-1} + \ldots + a^2/b^2 + a/b + 1)$
    $= cn(a^k/b^k + \ldots + a^2/b^2 + a/b + 1)$

Ex. 4 – Iteração (4)

- Então, para $k = \log_b n$
  - $T(n) = cn(a^k/b^k + \ldots + a^2/b^2 + a/b + 1)$

- E se a = b?
  - $T(n) = cn(k + 1)$
    $= cn(\log_b n + 1)$
    $= \Theta(n \log n)$

Ex. 4 – Iteração (5)

- E se a < b?

$T(n) = cn(a^k/b^k + \ldots + a^2/b^2 + a/b + 1)$

  - Lembrando que:

$$\Sigma(x^k + x^{k-1} + \ldots + x + 1) = (x^{k+1} - 1)/(x-1)$$

  - Temos:

$$\frac{a^k}{b^k} + \frac{a^{k-1}}{b^{k-1}} + \cdots + \frac{a}{b} + 1 = \frac{(a/b)^{k+1} - 1}{(a/b) - 1} = \frac{1 - (a/b)^{k+1}}{1 - (a/b)} < \frac{1}{1 - a/b}$$

  - Então:

$$T(n) = cn \cdot \Theta(1) = \Theta(n)$$

Ex. 4 – Iteração (6)

- E se a > b?

$T(n) = cn(a^k/b^k + \ldots + a^2/b^2 + a/b + 1)$

$$\frac{a^k}{b^k} + \frac{a^{k-1}}{b^{k-1}} + \cdots + \frac{a}{b} + 1 = \frac{(a/b)^{k+1} - 1}{(a/b) - 1} = \Theta\left((a/b)^k\right)$$

- $T(n) = cn \cdot \Theta(a^k / b^k)$

  $= cn \cdot \Theta(a^{\log n} / b^{\log n}) = cn \cdot \Theta(a^{\log n} / n)$

  como, $a^{\log n} = n^{\log a}$

  $= cn \cdot \Theta(n^{\log a} / n) = \Theta(cn \cdot n^{\log a} / n)$

  $= \Theta(n^{\log a})$

Ex. 4 – Iteração (7)

- Portanto, finalmente …

$$T(n) = \begin{cases} \Theta(n) & a < b \\ \Theta(n \log_b n) & a = b \\ \Theta\left(n^{\log_b a}\right) & a > b \end{cases}$$

Exercícios sobre resolução de equações de recorrência Parte I

Analisando custo de programas recursivos

Determine a função de custo

```
Faz-Algo(int n) {
If(n > 0) {
    FOR(x=0; x< n; x++) {
        Faz-Algo(n-1);
    }
}
Else {
   Operação-unitária
 }
}
```

Determine a função de custo

```
Faz-Algo(int n) {
If(n > 0) {
    Faz-Algo(n-1);
   FOR(x=0; x< n; x++) {
          Operação-unitária
   }
 }
}
```

Determine a função de custo

```
Faz-Algo(int n) {
 If(n > 1) {
     Faz-Algo(n/3);
     Faz-Algo(n/3);
     FOR(x=0; x< n; x++) {
          Operação-unitária
    }
 }
}
```

Árvores de Recursão

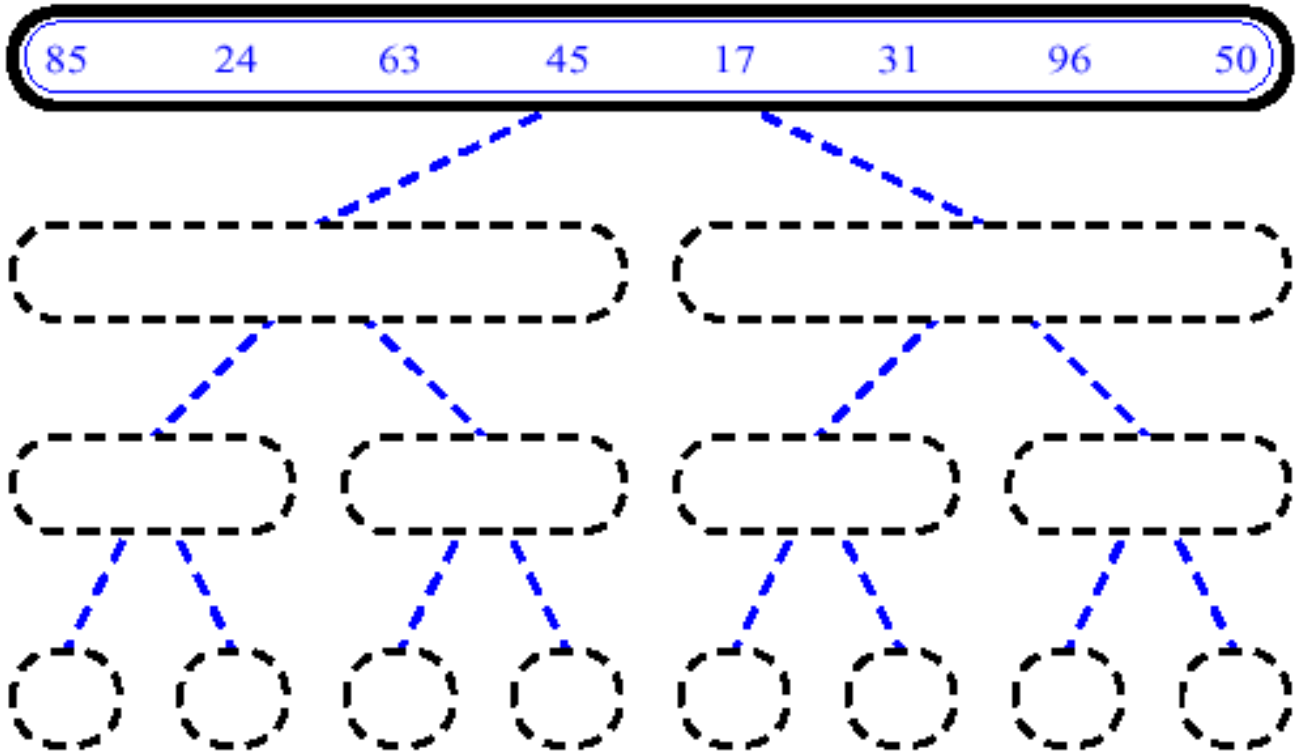
Árvores de Recursão

- Cada nodo representa o custo de um sub-problema individual em algum ponto da iteração das invocações recursivas
- Soma-se os custos em cada nível de recursão
- O custo total é obtido somando-se os custos de cada nível de recursão

Árvores de Recursão (2)

- Útil quando a recorrência é usada para problemas de divisão e conquista
- Você usa para fazer estimativas e depois pode provar por indução que está certo
  - Pode-se ignorar os tetos e pisos e utilzar coeficientes implícitos $c > 0$

Árvore de recursão:mergesort



Teorema Mestre

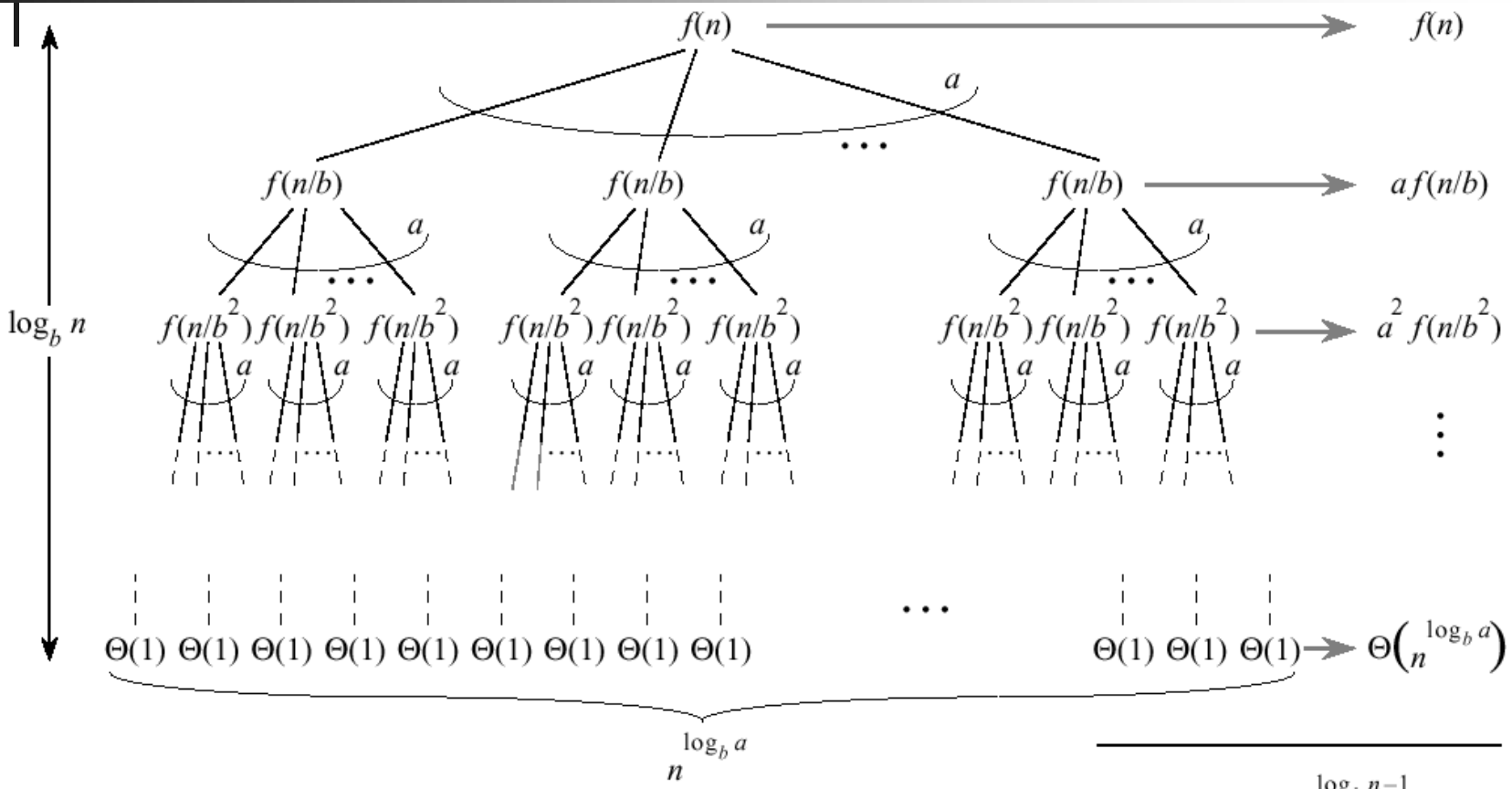
Teorema Mestre

- Usado para a solução de recorrências da forma

$$T(n) = aT(n/b) + f(n)$$

- $a \geq 1$, $b > 1$, e f assintóticamente positiva;
- af(n/b)<= kf(n) (k menor que 1 e n grande)
- Sendo $T(n)$ o tempo de execução do algoritmo, podemos dizer que
  - *Sub-problemas de tamanho n/b* são resolvidos recursivamente cada um em tempo $T(n/b)$
  - $f(n)$ é o custo de dividir o problema e combinar seus resultados. No Mergesort temos: $T(n) = 2T(n/2) + \Theta(n)$

Teorema Mestre (2)



- *Os sub problemas são divididos em* a *partes.*
- *Existirão* $\log_b n$ *níveis*
- *Ocorrerão* $a^{\log_b n} = n^{\log_b a}$ *folhas*

Teorema Mestre (5)

- Três casos comuns:
  - Tempo de execução é dominado pelo custo nas folhas
  - Tempo de execução é uniformemente distribuído pela árvore
  - Tempo de execução é dominado pelo custo na raiz
- Assim, para resolver a recorrência, teremos de caracterizar o termo dominante
- Para cada caso comparar

$$f(n) \qquad O(n^{\log_b a})$$

Resumo do Teorema Mestre

- Seja uma recorrência da forma

$$T(n) = aT(n/b) + f(n)$$

1. $f(n) = O(n^{\log_b a - \varepsilon}), \varepsilon > 0 \quad \Rightarrow \quad T(n) = \Theta(n^{\log_b a})$
2. $f(n) = \Theta(n^{\log_b a}) \quad \Rightarrow \quad T(n) = \Theta(n^{\log_b a} \lg n)$
3. $f(n) = \Omega(n^{\log_b a + \varepsilon}), \varepsilon > 0 \quad \Rightarrow \quad T(n) = \Theta(f(n))$

   sendo $a.f(n/b) \leq c.f(n), c < 1$

Estratégia

- Identificar *a*, *b* e *f (n)*
- Determinar $n^{\log_b a}$
- Comparar *f (n)* e $n^{\log_b a}$ assintóticamente
- De acordo com o caso, aplicar a regrar correspondente

Exemplo

- $T(n) = 9T(n/3) + n$
  - $a=9, b=3, f(n) = n$
  - $n^{\log_b a} = n^{\log_3 9} = n^2$
  - Com $f(n) = O(n^{\log_3 9 - \varepsilon})$, onde $\varepsilon=1$, o caso 1 se aplica:

$$T(n) = \Theta\left(n^{\log_b a}\right) \text{ quando } f(n) = O\left(n^{\log_b a - \varepsilon}\right)$$

  - Portanto $T(n) = \Theta(n^2)$

Exemplo: Mergesort

$$T(n) = 2T(n/2) + \Theta(n)$$

$$a = 2,\ b = 2;\ n^{\log_b a} = n^{\log_2 2} = n$$

Como $f(n) = \Theta(n)$

Temos o caso 2 :

$$f(n) = \Theta(n^{\log_b a}) \quad T(n) = \Theta(n^{\log_b a} \lg n)$$

$$T(n) = \Theta\left(n^{\log_b a} \lg n\right) = \Theta\left(n \lg n\right)$$

Método Aniquilador (ou resolvendo EDOs...)

Método aniquilador

- Para resolver equações na forma:

$$t(n) = a_{n-1}t(n-1) + a_{n-2}t(n-2) + \cdots + a_0 t(0) + \alpha(n)$$

- Se α(n)=0, t(n) é homogênea
- Grau d de t(n) é dado pelo maior a_{n-d} não nulo.

Aniquilando sequencias

$$\begin{cases} t(0) = 0 \\ t(1) = 1 \\ n > 1 : t(n) = t(n-1) + t(n-2) \end{cases}$$

Passo 1: Encontrar Aniquilador

$$
\begin{cases}
Fn = < 0,1,t(2),\ldots,t(n) > \\
EFn = < 1,t(2),\ldots,t(n+1) > \\
E^2Fn = < t(2) = t(1) + t(0),\ldots,t(n+2) = t(n+1) + t(n) > \\
(E^2 - E - 1)Fn = < t(1) + t(0) - 1 - 0,\ldots,t(n+1) + t(n) - t(n+1) - t(n) > \\
< 0,0,\ldots,0 >
\end{cases}
$$

Passo 2: Fatorar o aniquilador

$$\left(E^2 - E - 1\right) = 0$$

$$\left(E - r1\right)\left(E - r2\right) = 0$$

Passo 2: Encontrar as raizes da equação

$$
\begin{pmatrix}
r = \dfrac{-b \pm \sqrt[2]{b^2 - 4ac}}{2a} \\
E^2 - E - 1 \\
r = \dfrac{-(-1) \pm \sqrt[2]{1^2 - 4(-1)(1)}}{2(1)} \\
r = \dfrac{1 \pm \sqrt[2]{5}}{2} \\
r1 = \dfrac{1 + \sqrt[2]{5}}{2} \\
r2 = \dfrac{1 - \sqrt[2]{5}}{2}
\end{pmatrix}
$$

Passo 3: Forma geral da eq

Raizes distintas:

$$t(n) = c_1 r_1^{\,n} + c_2 r_2^{\,n}$$

Raizes iguais (DELTA = 0):

$$t(n) = c_1 r_1^{\,n} + c_2 n r_2^{\,n}$$

Passo 4:Usar casos base para encontrar constantes

$$t(n) = c_1 r_1^{\,n} + c_2 r_2^{\,n}$$

$$t(1) = c_1 r_1^{\,1} + c_2 r_2^{\,1} = 1$$

$$t(0) = c_1 r_1^{\,0} + c_2 r_2^{\,0} = 0$$

$$c1 = \frac{1}{\sqrt[2]{5}}$$

$$c2 = -\frac{1}{\sqrt[2]{5}}$$

Passo 3: Forma geral da eq

$$t(n) = \frac{1}{\sqrt[2]{5}}\left(\frac{1+\sqrt[2]{5}}{2}\right)^n - \frac{1}{\sqrt[2]{5}}\left(\frac{1-\sqrt[2]{5}}{2}\right)^n$$

$$t(n) = 0{,}447213595499958(1{,}618033988749895)^n - 0{,}447213595499958(-0{,}618033988749895)^n$$

$$\begin{cases} t(0) = 0 \\ t(1) = 1 \\ n > 1 : t(n) = 2t(n-1) + 3t(n-2) \end{cases}$$

$$(E^2 - 2E - 3) = 0$$

Raizes: 3 e -1

$$t(n) = c_1 r_1^{\,n} + c_2 r_2^{\,n}$$

$$t(1) = c_1 3^1 + c_2(-1)^1 = 1$$

$$t(0) = c_1 3^0 + c_2(-1)^0 = 0$$

$$c_1 + c_2 = 0$$

$$c_1 = -c_2$$

$$c_1 = \frac{1}{4} = 0{,}25$$

$$c_2 = -\frac{1}{4} = 0{,}25$$

$$t(n) = c_1 r_1^{\,n} + c_2 r_2^{\,n}$$

$$t(n) = 0{,}25 x 3^n - 0{,}25 x (-1)^n$$

Método da Substituição

Conceitos básicos

- Dois passos
  - Estimar a forma da solução
  - Usar indução matemática para determinar se a solução se aplica
- Útil quando é fácil estimar a forma da solução
- Pode ser utilizado para estabelecer limites superiores ou inferiores das recorrências

Exemplo – Substituição

- Resolver a recorrência

$$T(n) = 2T(\lfloor n/2 \rfloor) + n$$

T(n) = O(n lg n) ?

Exemplo – Substituição (2)

- Determinar um limite superior para

$$t(n) = \begin{cases} 2T(\lfloor n/2 \rfloor) + n & n > 1 \\ 1 & n = 1 \end{cases}$$

- Se T(n) = O(n lg n)
- Provar que T(n) $\leq$ cn lg n para algum c usando indução e para todo $n \geq n_0$

Exemplo – Substituição (3)

- Base da indução: mostrar que a inequação é válida para algum n suficientemente pequeno
  - Se n= 1 → T(1) ≤ c * 1 * log 1 = 0 !!!
  - No entanto T(1) = 1
  - Mas

n=2 → T(2) =2T(1) + 2 = 4 e *cn* lg *n* = c * 2 * lg 2 = 2c

n=3 → T(3)=2T(1) + 3 = 5 e *cn* lg *n* = c * 3 * lg 3

Exemplo – Substituição (4)

- Pode-se partir de T(2)=4 ou T(3)=5 usando qualquer $c \geq 2$
- Válido de acordo com a notação assintótica: $T(n) \leq cn \lg n$ *para* $n \geq n_0$
- Truque: estender as condições de contorno para fazer a hipótose indutiva valer para pequenos valores de n

Exemplo – Substituição (5)

- Hipótese indutiva:
  - Assumir a inequação para $\lfloor n/2 \rfloor$

$$T(\lfloor n/2 \rfloor) \leq c\lfloor n/2 \rfloor \lg \lfloor n/2 \rfloor)$$

Exemplo – Substituição (5)

- Indução : A inequação é válida para n

$$
\begin{aligned}
T(n) &= 2T(\lfloor n/2 \rfloor) + n \\
&\le 2(c\lfloor n/2 \rfloor \lg \lfloor n/2 \rfloor)) + n \\
&\le cn\lg(n/2) + n \\
&= cn(\lg n - \lg 2) + n \\
&\le cn\lg n - cn + n \\
&\le cn\lg n
\end{aligned}
$$

(é valida para c ≥1)

$T(\lfloor n/2 \rfloor) \le c\lfloor n/2 \rfloor \lg \lfloor n/2 \rfloor)$

Sobre Estimativas

- Requer experiência e criatividade
- Pode-se utilizar árvores de recursão
- Se a recursão é similar a uma outra com a qual se está familiarizado, é razoável tentar uma solução similar $T(n) = 2T(\lfloor n/2 \rfloor + 17) + n$
  - T(n) = O(*n* lg *n*) ➔ Porque?
  - O termo adicional (17) não afetará substancialmente a solução da recorrência para um n "grande"

Sobre Estimativas (2)

- As vezes a estimativa está correta mas as contas "não fecham" na hora da indução
  - A hipótese indutiva suficientemente forte !!
  - Revisar a estimativa subtraindo um termo de menor ordem assintótica

Sobre Estimativas (3)

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$$ é O(n) ???

Mostrar que T(n) ≤ cn

$$T(n) \le c(\lfloor n/2 \rfloor) + c(\lceil n/2 \rceil) + 1 = cn + 1$$ Parece incorreto !!

Nova tentativa T(n) ≤ cn-b

$$T(n) \le c(\lfloor n/2 \rfloor - b) + c(\lceil n/2 \rceil - b) + 1 = cn - 2b + 1$$

$$T(n) \le cn - b$$

(para todo b ≥1)

Troca de Variáveis

$$T(n) = 2T(\sqrt{n}) + \lg n$$

$$T(2^m) = 2T(2^{m/2}) + m$$

$$S(m) = 2S(m/2) + m$$

$$T(n) = T(2^m) = S(m)$$

$$= O(m \lg m) = O(\lg n \lg \lg n)$$

$m = \lg n$

$S(m)=T(2^m)$

S(m)=O(m lg m)